

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 6

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SYED MAHATASIN HUSSAIN.

STUDENT ID : 23101140

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 02 - 09 - 2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Given the following processes with their **Arrival Time (AT)**, **Burst Time (BT)**, and **Priority**:

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	3	3
P2	1	4	2
P3	2	6	4
P4	3	4	6
P5	5	2	10

Implement the following CPU Scheduling Algorithms:

- **Priority Non-Preemptive Scheduling**
- **Priority Preemptive Scheduling**

Assume that a **smaller priority number indicates a higher priority**.

For each algorithm, calculate the following for every process:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**

Also, calculate:

- **Average Turnaround Time (Average TAT)**
- **Average Waiting Time (Average WT)**

Finally, display the results for all processes and **compare the Average TAT and Average WT of both Preemptive and Non-Preemptive Priority Scheduling algorithms to determine which algorithm performs better for the given processes.**

2. Source Code:

```
# Priority Non-Preemptive

process = ["p1", "p2", "p3", "p4", "p5"]
at = [0,1,2,3,5]
bt = [3,4,6,4,2]
priority = [3,2,4,6,10]

n = len(process)

ct = [0] * n
wt = [0] * n
done = [0] * n
tat = [0] * n

time = 0
completed = 0

while completed < n:

    x = -1
    for i in range(n):

        if at[i] <= time and done[i] == 0:

            if x == -1 or priority[i] < priority[x]:
                x = i

    if x == -1:
        time += 1

    else:
        time = time + bt[x]

        ct[x] = time
```

```
        done[x] = 1
        completed += 1


for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]


print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i],
"\t", tat[i], "\t", wt[i])

non_P_avg_tat = sum(tat)/n
non_p_avg_wt = sum(wt)/n

print("\nAverage TAT =", non_P_avg_tat)
print("Average WT =", non_p_avg_wt)
print("\n")


# Priority Preemptive

process = ["p1", "p2", "p3", "p4", "p5"]
at = [0,1,2,3,5]
bt = [3,4,6,4,2]
priority = [3,2,4,6,10]


n = len(process)

ct = [0] * n
wt = [0] * n
tat = [0] * n
```

```
tq= 1
remaining = bt.copy()
time = 0
completed = 0

while completed < n:

    x = -1
    for i in range(n):

        if at[i] <= time and remaining[i] > 0:

            if x == -1 or priority[i] < priority[x]:
                x = i

    if x == -1:
        time += 1

    else:
        if remaining[x] > tq:
            time += tq
            remaining[x] -= tq

        else:
            time += remaining[x]
            remaining[x] = 0
            ct[x] = time
            completed += 1


for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]


print("Process\tAT\tBT\tCT\tTAT\tWT")
```

```
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i],
"\t", tat[i], "\t", wt[i])

p_avg_tat = sum(tat)/n
p_avg_wt = sum(wt)/n

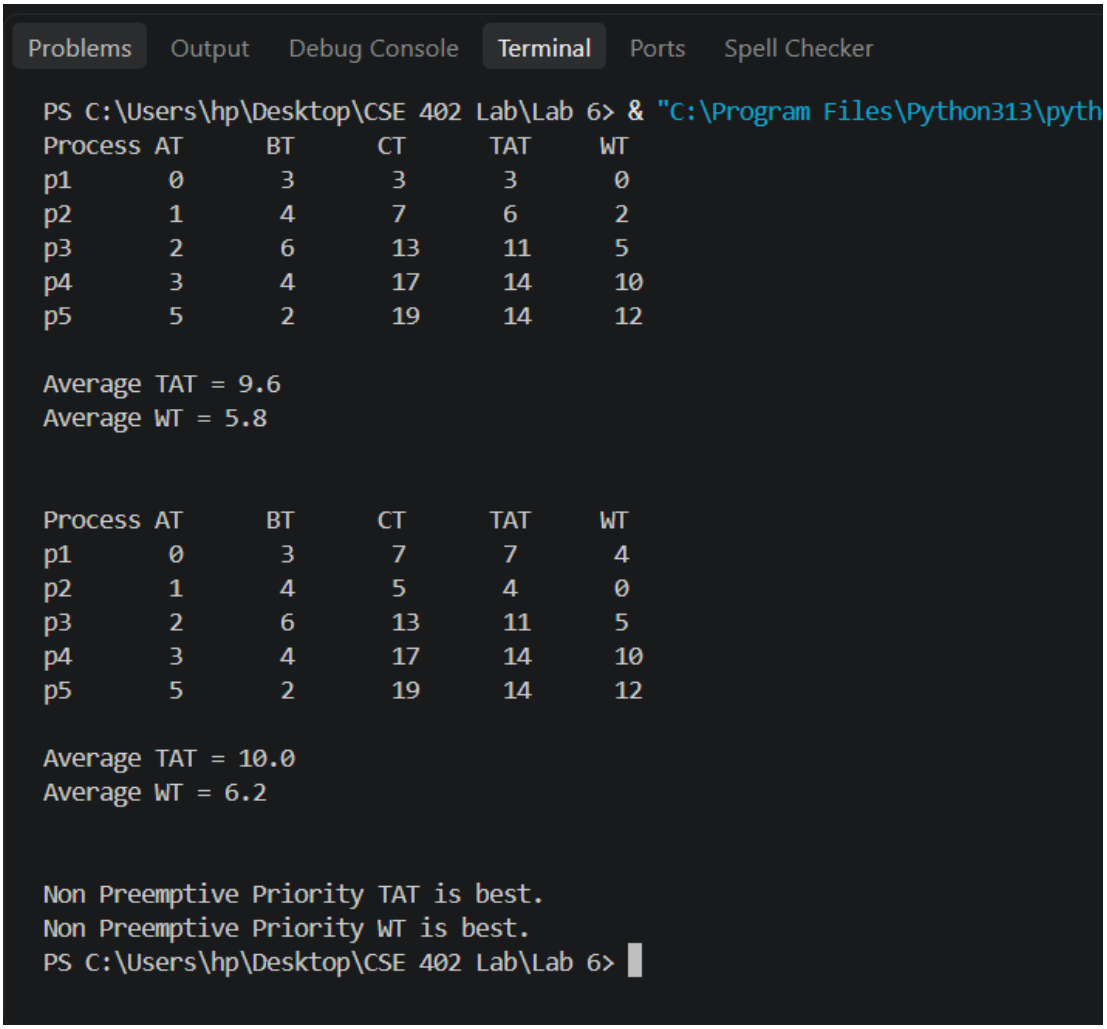
print("\nAverage TAT =", p_avg_tat)
print("Average WT =", p_avg_wt)
print("\n")



if non_P_avg_tat < p_avg_tat :
    print ("Non Preemptive Priority TAT is best.")
else:
    print("Preemptive Priority TAT is best")

if non_p_avg_wt < p_avg_wt :
    print ("Non Preemptive Priority WT is best.")
else:
    print("Preemptive Priority WT is best")
```

3. Output:



4. Discussion:

In this experiment, **Priority Non-Preemptive** and **Priority Preemptive** CPU scheduling algorithms were implemented and compared using the same set of processes. The performance was evaluated based on **Average Turnaround Time (TAT)** and **Average Waiting Time (WT)**.

The **Priority Non-Preemptive algorithm** produced an Average TAT of **9.6** and an Average WT of **5.8**. In this algorithm, once a process starts execution, it continues until completion without interruption.

On the other hand, the **Priority Preemptive algorithm** produced an Average TAT of **10.0** and an Average WT of **6.2**. In this algorithm, a running process can be interrupted if another higher-priority process becomes available.

By comparing the results, the Non-Preemptive Priority algorithm has lower Average TAT and Average WT than the Preemptive Priority algorithm. Therefore, **Non-Preemptive Priority Scheduling performs better for this particular set of processes**.

5. Conclusion:

The experiment successfully implemented and compared **Preemptive and Non-Preemptive Priority Scheduling algorithms**. Based on the experimental results, **Non-Preemptive Priority Scheduling is the best-performing algorithm for the given processes** because it achieved the lower Average TAT of **9.6** and Average WT of **5.8**. In comparison, Preemptive Priority Scheduling produced an Average TAT of **10.0** and Average WT of **6.2**.

Therefore, according to the comparison of **Average TAT and Average WT**, **Non-Preemptive Priority Scheduling is better for this particular dataset**. However, Preemptive Priority Scheduling can be more useful when a newly arriving high-priority process needs immediate CPU access.

6. GitHub:

https://github.com/Rizon40/CSE-402-Operating-System-Lab